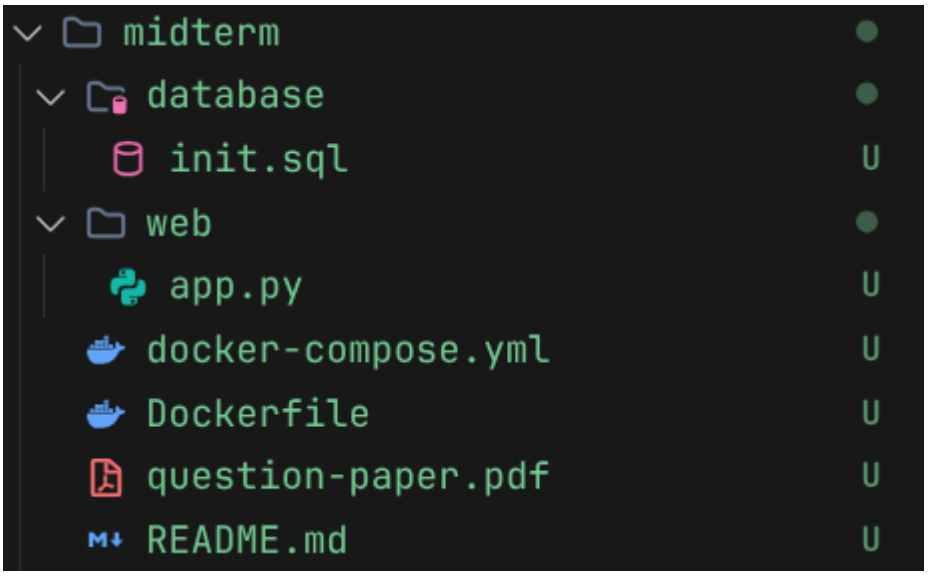
MIDTERM EXAMINATION

20CYS402 - DISTRIBUTED SYSTEMS AND CLOUD COMPUTING

PART B : DOCKERIZED WEB AUTHENTICATION APPLICATION

CB.SC.U4CYS23021
K.C.K.SANJAI

FILE STRUCTURE



CODE FILES

Init.sql

```sql
create table users (
   id serial primary key,
   username varchar(50) unique not null,
   password varchar(50) not null
);
```

```
insert into users(username, password) values('sanjai',
'sanjai235');
```

app.py

Python

```python
import os
import psycopg2
from flask import Flask, request, render_template_string

app = Flask(__name__)

PAGE = """
   <h1>Log in</h1>
   <form method="post">
       Username: <input name="username">
       <br><br>
       Password: <input name="password" type="password">
       <br><br>
       <button type="submit">login</button>
   </form>
   <p> {{ msg }} </p>
"""

def get_db():
   return psycopg2.connect(
       host=os.environ.get("host", "db"),
       dbname=os.environ.get("dbname", "midterm_db"),
       user=os.environ.get("username", "midterm_user"),
       password=os.environ.get("password", "midterm_password"),
   )

@app.route("/", methods=["GET", "POST"])
def login():
   msg = ""
   if request.method == "POST":
```

```
        conn = get_db()
        cur = conn.cursor()
        cur.execute(
            "SELECT 1 FROM users WHERE username = %s AND password
= %s",
            (request.form["username"], request.form["password"]),
        )
        msg = "Login successful" if cur.fetchone() else "Invalid
username or password"
        conn.close()
    return render_template_string(PAGE, msg=msg)

app.run(host="0.0.0.0", port=5000)
```

(The render_template_string is used to directly display a HTML page from a python string)

Dockerfile

```
None
FROM python:3.12-slim

ENV ROLLNUMBER="cb.sc.u4cys23021"
ENV NAME="K.C.K.SANJAI"

LABEL EMAIL="cb.sc.u4cys23021@cb.students.amrita.edu"
LABEL PRODUCTION_VERSION="1.0"

WORKDIR /midterm-exam

RUN pip install --no-cache-dir flask psycopg2-binary

COPY web/ /midterm-exam/

EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

Dockercompose.yml

```
None
services:
 db:
   image: postgres:16
   environment:
     POSTGRES_DB: midterm_db
     POSTGRES_USER: midterm_user
     POSTGRES_PASSWORD: midterm_pass
   volumes:
     - ./database:/docker-entrypoint-initdb.d

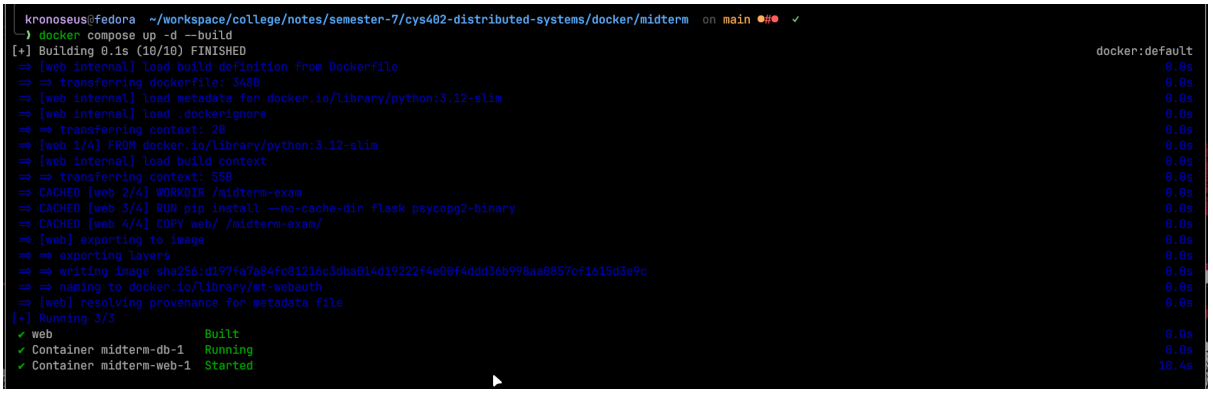
 web:
   build: .
   image: mt-webauth
   environment:
     host: db
     username: midterm_user
     password: midterm_pass
     dbname: midterm_db
   ports:
     - "5000:5000"
   volumes:
     - ./web:/midterm-exam
   depends_on:
     - db
```

What the `volumes: - ./database:/docker-entrypoint-initdb.d` line does:
Inside the /database i have the init.sql so that line copies it to the docker-entrypoint-initdb.d of the image, which is a entry point to set up databases (i.e the functionality of init.sql)

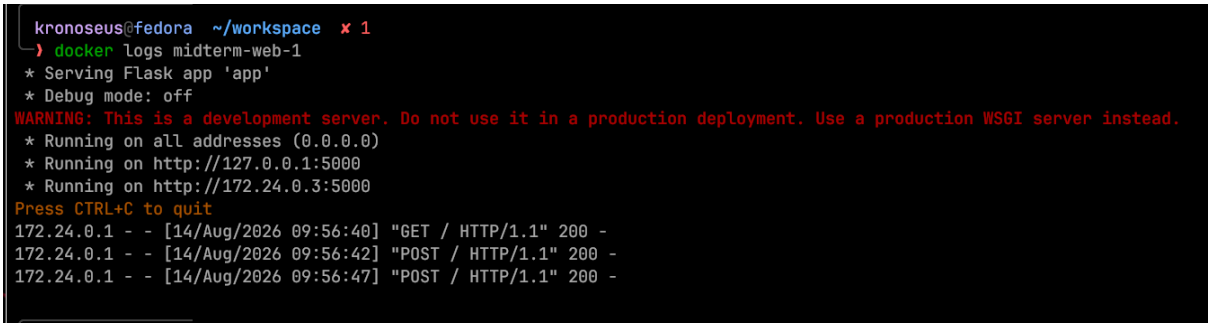
(the overall explanation is below, i explained this alone explicitly because i wanted to convey my intent)

OUTPUTS

Running the compose



Checking out the logs (of the service app)



Inspecting the image to verify ENV and LABEL are there

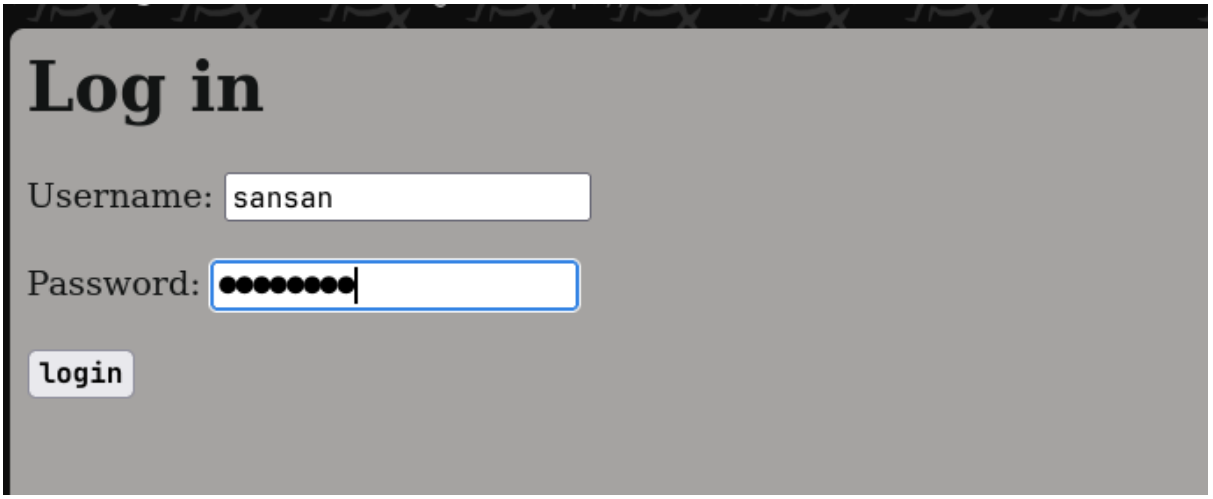


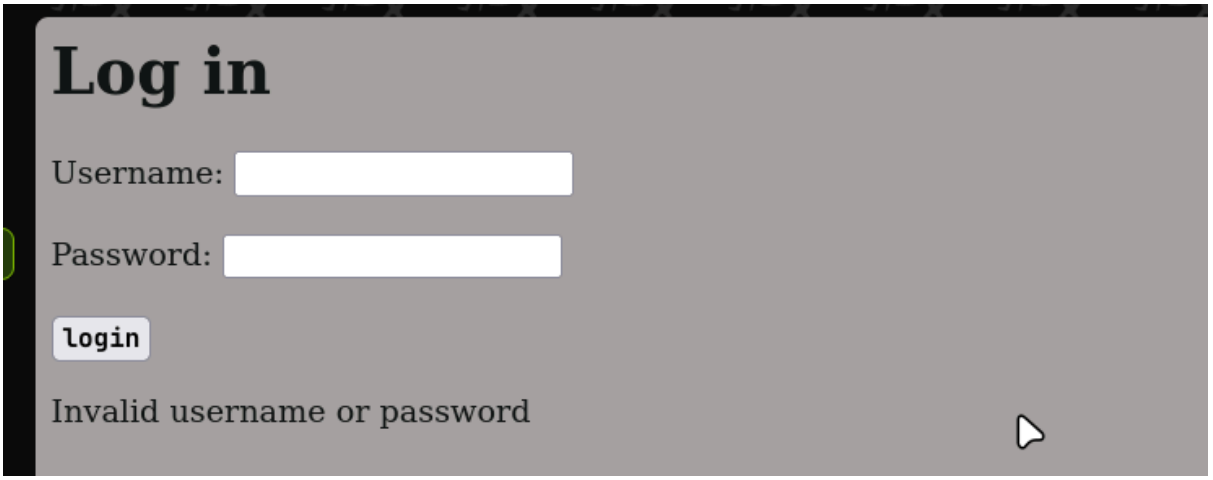


Checking out the localhost at port 5000

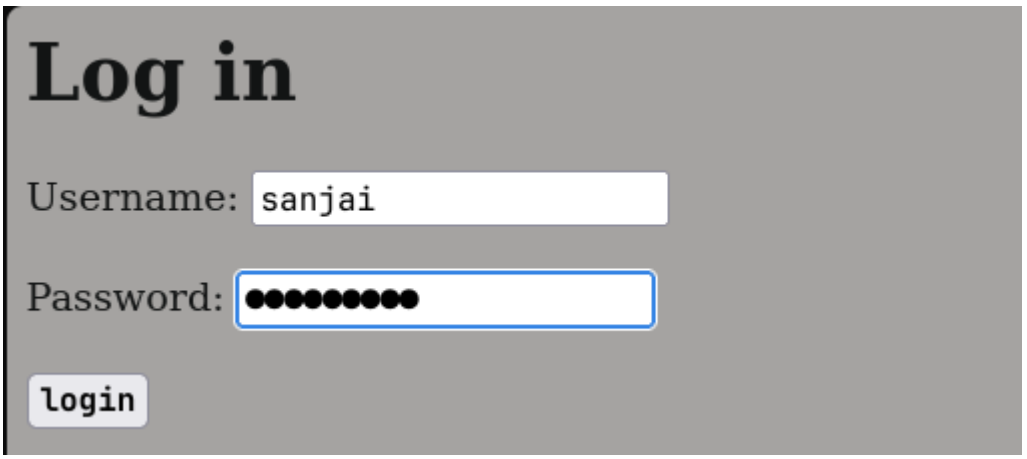


Giving wrong credentials (the sample correct one is: username: “sanjai”, password: “sanjai235”)



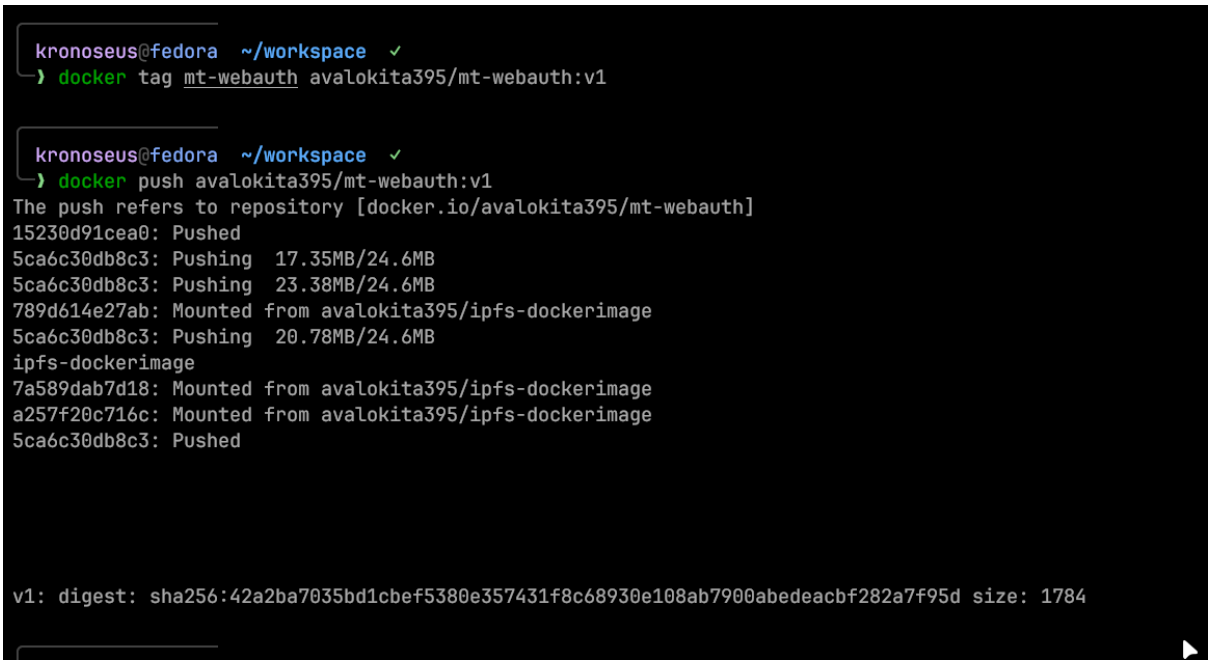


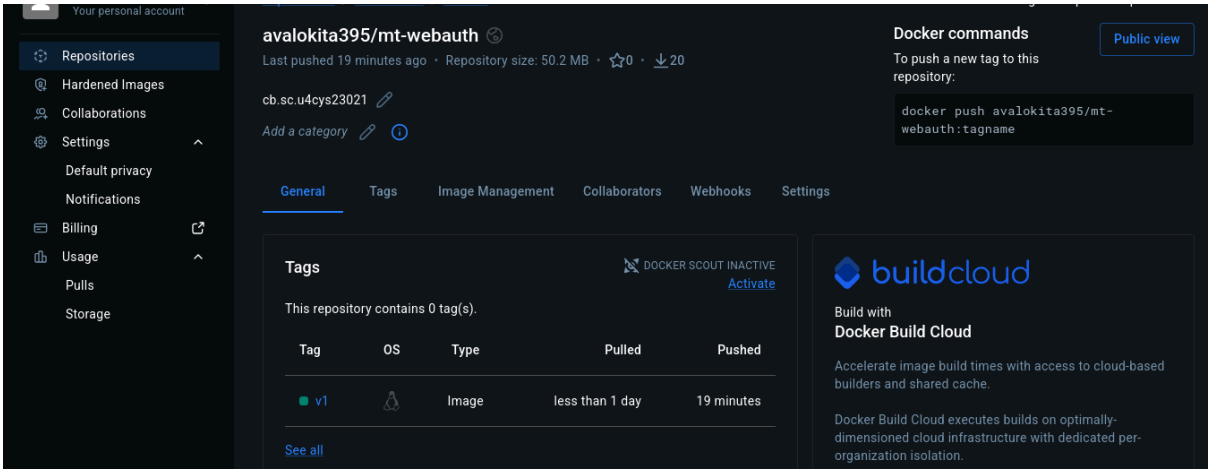
Giving the correct (sample) credentials





Pushing to dockerhub





EXPLANATION:

- In this task, I created a small login system using Docker, Flask, and PostgreSQL.
- I wrote the Dockerfile using python:3.12-slim , installed Flask and psycopg2-binary , and added my roll number and name as environment variables.
- Then I used docker-compose.yml to run two services together: one for the PostgreSQL database and one for the Flask web app.
- I created an init.sql file to make a users table and insert one sample user record.
- In the Flask app, I added a simple login form and wrote the logic to connect to the database, check whether the entered username and password match a row in the users table, and show either success or invalid login.
- I also exposed port 5000 and added the basic run, inspect, and stop commands in the README so the full setup can be tested easily.
- Finally i tagged the local image to the dockerhub repository name version 1 (v1) and pushed it